

Toy Blockchain CLI Project Report

Project Title

Toy Blockchain Command Line Application Using Go

Abstract

This project is a simple blockchain command line application developed using the Go programming language. The main purpose of this project is to demonstrate the basic internal working concepts of a blockchain system in a simple and understandable way.

The application allows users to create a blockchain, add transactions, mine blocks, view blockchain data, check account balances, validate the blockchain, view pending transactions, and deliberately tamper with old data to test blockchain security. The blockchain data is stored in a JSON file, allowing the data to persist even after the program stops.

This project does not aim to build a real cryptocurrency system. Instead, it focuses on the core educational concepts of blockchain, such as blocks, transactions, hashing, proof of work, previous hash linking, chain validation, and tamper detection.

Table of Contents

1. Introduction
 2. Problem Statement
 3. Project Objectives
 4. Scope of the Project
 5. Tools and Technologies Used
 6. Project Folder Structure
 7. System Overview
 8. Main Features
 9. Blockchain Concepts Used
 10. System Architecture
 11. Explanation of Main Program
 12. Explanation of Commands
 13. Data Flow of the System
 14. Testing and Validation
 15. Example Usage
 16. Results and Observations
 17. Limitations
 18. Future Improvements
 19. Conclusion
-

1. Introduction

Blockchain is a distributed data storage technology where data is stored in blocks. Each block is connected to the previous block using a cryptographic hash. Because of this hash connection, changing data in an old block affects all following blocks. This makes blockchain suitable for systems that require data integrity, transparency, and tamper detection.

This project implements a simplified blockchain system as a command line interface. The user interacts with the blockchain through terminal commands. The system supports adding transactions, mining transactions into blocks, viewing balances, validating the blockchain, and testing tampering.

The project is implemented using Go because Go is simple, fast, strongly typed, and suitable for building command line tools and backend systems.

2. Problem Statement

The main problem addressed by this project is understanding how blockchain works internally. Many blockchain systems are complex because they include peer-to-peer networking, cryptographic wallets, smart contracts, distributed consensus, and advanced security mechanisms.

For learning purposes, this project simplifies the blockchain concept and focuses only on the most important internal mechanisms:

- How transactions are created.
- How transactions are stored before mining.
- How a block is created.
- How proof of work is performed.
- How blocks are connected using hashes.
- How blockchain validation detects tampering.
- How account balances can be calculated from transactions.

3. Project Objectives

The main objectives of this project are:

1. To implement a simple blockchain using Go.
2. To create a command line interface for interacting with the blockchain.
3. To add transactions into a pending transaction pool.
4. To mine pending transactions into new blocks.
5. To use proof of work during block mining.
6. To store blockchain data in a JSON file.
7. To validate the blockchain after each operation.
8. To calculate account balances from confirmed transactions.
9. To deliberately tamper with blockchain data and observe validation failure.
10. To understand how blockchain protects data integrity.

4. Scope of the Project

This project includes the basic blockchain operations required for educational demonstration.

Included in the Scope

- Blockchain initialization.
- Genesis block creation.
- Transaction creation.
- Pending transaction pool.
- Block mining.
- Proof-of-work difficulty.
- Blockchain printing.
- Blockchain validation.
- Balance calculation.
- Pending transaction viewing.
- Tamper testing.
- JSON-based data storage.
- Unit testing for important packages.

Not Included in the Scope

This project does not include:

- Real cryptocurrency wallets.
- Digital signatures.
- Peer-to-peer network.
- Distributed mining.
- Smart contracts.
- Real financial transactions.
- Merkle trees.
- User authentication.
- Web interface.
- Database integration.

Therefore, this project should be considered a learning-based toy blockchain, not a production blockchain.

5. Tools and Technologies Used

Tool / Technology	Purpose
Go	Main programming language
Go CLI	Running and testing the program
JSON	Storing blockchain data
Go <code>flag</code> package	Reading command line options
Go <code>os</code> package	Handling terminal arguments and file errors
Go <code>fmt</code> package	Printing output

Tool / Technology	Purpose
Go testing package	Unit testing
Git / GitHub	Version control and project submission

6. Project Folder Structure

The project is organized into separate packages. This improves readability and maintainability.

```
toy-blockchain/
├── block/
│   ├── block.go
│   └── block_test.go
├── chain/
│   ├── chain.go
│   └── chain_test.go
├── cmd/
│   └── toychain/
│       └── main.go
├── data/
│   └── chain.json
├── internal/
│   └── transaction/
│       └── transaction.go
├── ledger/
│   └── ledger.go
├── storage/
│   ├── storage.go
│   └── storage_test.go
├── go.mod
└── README.md
```

Explanation of Main Folders

Folder	Description
block	Contains block-related logic
chain	Contains blockchain operations such as mining and validation
cmd/toychain	Contains the command line application

Folder	Description
<code>internal/transaction</code>	Contains transaction-related structures
<code>storage</code>	Handles saving and loading blockchain data
<code>data</code>	Stores the blockchain JSON file
<code>ledger</code>	Can be used for balance or ledger-related logic

7. System Overview

The system works as a command line application. The user enters a command in the terminal. The `main.go` file reads the command and calls the correct function.

For example:

```
go run ./cmd/toychain add -from Alice -to Bob -amount 50
```

The system then:

1. Reads the command.
2. Loads the blockchain from `data/chain.json`.
3. Creates a transaction.
4. Adds it to the pending transaction pool.
5. Saves the updated blockchain.
6. Prints a success message.

The blockchain is not stored only in memory. It is saved into a JSON file. Therefore, the chain can be reused later.

8. Main Features

8.1 Create a New Blockchain

The `init` command creates a new blockchain file.

```
go run ./cmd/toychain init
```

This creates the genesis block and saves the blockchain into `data/chain.json`.

8.2 Add a Transaction

The `add` command adds a transaction to the pending transaction pool.

```
go run ./cmd/toychain add -from Alice -to Bob -amount 50
```

This transaction is not immediately added to the blockchain. It waits in the pending pool until mining is performed.

8.3 Mine a Block

The **mine** command mines pending transactions into a new block.

```
go run ./cmd/toychain mine
```

Mining performs proof of work and adds a new block to the blockchain.

8.4 Print the Blockchain

The **print** command displays all blocks in the blockchain.

```
go run ./cmd/toychain print
```

This shows block height, timestamp, previous hash, nonce, hash, and transactions.

8.5 Validate the Blockchain

The **validate** command checks whether the blockchain is valid.

```
go run ./cmd/toychain validate
```

If no data has been changed, the output is:

```
Chain valid
```

If old data has been changed, the output becomes:

```
Chain invalid
```

8.6 Show Balances

The **balances** command calculates account balances.

```
go run ./cmd/toychain balances
```

Balances are calculated from confirmed mined transactions.

8.7 Show Pending Transactions

The **pending** command shows transactions waiting to be mined.

```
go run ./cmd/toychain pending
```

8.8 Tamper with Data

The **tamper** command deliberately modifies old transaction data.

```
go run ./cmd/toychain tamper -block 1 -tx 0 -amount 999
```

This is used to demonstrate blockchain tamper detection.

9. Blockchain Concepts Used

9.1 Block

A block is a container that stores transactions and metadata.

A typical block contains:

- Block height.
- Timestamp.
- Previous block hash.
- Transactions.
- Nonce.
- Current block hash.

The block hash is calculated using block data. If the block data changes, the hash also changes.

9.1.1 Deterministic SHA-256 Block Hashing

This project uses deterministic SHA-256 hashing to generate the hash of each block. Deterministic hashing means that the same block data will always produce the same hash when the same field order is used.

The hash is generated by converting important block fields into a fixed input format and then applying the SHA-256 algorithm.

Fields included in the block hash

The following block fields are included in the hash calculation in this exact order:

Order	Field
1	Block height / index
2	Timestamp
3	Previous block hash
4	Nonce
5	Transactions

Each transaction is also added to the hash input using a fixed field order:

Order	Transaction Field
1	From account
2	To account
3	Amount

The block's own current hash field is not included in the hash calculation. This is because the hash is the final result of the calculation. Including the hash field itself would create a circular dependency.

Hash input format

The block hash is calculated using the following logical format:

```
height | timestamp | previousHash | nonce | transactions
```

Each transaction is represented using this format:

```
from | to | amount
```

Example logical input:

```
1|2026-07-09T10:30:00Z|000abc...|45|Alice|Bob|25
```

After this deterministic input string is created, SHA-256 is applied to generate the final block hash.

Importance of documented field order

The field order is documented because the assignment requires the hash input fields and their order to be clear.

This also makes the hashing process testable. If the same block data is used in the same order, the same SHA-256 hash is produced. If any field value changes, the hash changes. Therefore, blockchain validation can detect tampering.

9.2 Transaction

A transaction represents value movement between accounts.

Example:

```
Alice -> Bob : 50
```

In this project, a transaction contains:

- Sender account.
- Receiver account.
- Amount.

The system supports simple account names such as **Alice**, **Bob**, and **FAUCET**.

9.3 Genesis Block

The genesis block is the first block in the blockchain.

It has height **0**.

It does not have a real previous block, so its previous hash is usually empty or a default value.

The genesis block is created when the user runs:

```
go run ./cmd/toychain init
```

9.4 Pending Transaction Pool

When a transaction is added, it is first stored in the pending transaction pool.

It is not immediately part of the blockchain.

Example:

```
Pending Transactions:  
Alice -> Bob : 50
```

When mining is performed, pending transactions are selected and included in a new block.

9.5 Mining

Mining is the process of creating a valid block.

In this project, mining means:

1. Taking pending transactions.
 2. Creating a new block.
 3. Linking it to the previous block.
 4. Trying different nonce values.
 5. Finding a hash that satisfies the difficulty.
 6. Adding the block to the blockchain.
 7. Removing mined transactions from the pending pool.
-

9.6 Proof of Work

Proof of work is a method used to make block creation computationally difficult.

In this project, proof of work is controlled using the difficulty value.

For example, if difficulty is 2, the block hash may need to start with two zeros:

```
00abc123...
```

The miner keeps changing the nonce until a valid hash is found.

9.7 Previous Hash Linking

Each block stores the hash of the previous block.

Example:

```
Block 0 Hash:      00abc...  
Block 1 PrevHash:  00abc...  
Block 1 Hash:      00def...  
Block 2 PrevHash:  00def...
```

This creates a chain of blocks.

If the data in Block 1 is changed, its hash changes. Then Block 2's previous hash no longer matches. Therefore, the chain becomes invalid.

9.8 Blockchain Validation

Validation checks whether the blockchain is still correct.

The validation process checks:

1. Whether each block hash matches its data.
2. Whether each block points to the correct previous block hash.
3. Whether proof of work is satisfied.
4. Whether old data has been modified.

This is how the project demonstrates blockchain security.

10. System Architecture

The system follows a simple layered architecture.

```
User Terminal
  |
  v
Command Line Interface
cmd/toychain/main.go
  |
  v
Blockchain Logic
chain package
  |
  v
Block and Transaction Logic
block package + transaction package
  |
  v
Storage Layer
storage package
  |
  v
JSON File
data/chain.json
```

Explanation

The user does not directly access the blockchain data file. The user interacts through CLI commands. The CLI calls the blockchain logic. The blockchain logic modifies blocks and transactions. Finally, the storage package saves the updated blockchain into a JSON file.

11. Explanation of Main Program

The main command line program is located in:

```
cmd/toychain/main.go
```

This file is the entry point of the application.

It imports the required packages:

```
import (  
    "errors"  
    "flag"  
    "fmt"  
    "os"  
    "sort"  
  
    "toy-blockchain/chain"  
    "toy-blockchain/internal/transaction"  
    "toy-blockchain/storage"  
)
```

Main Responsibility of `main.go`

The `main.go` file is responsible for:

- Reading user commands.
- Parsing command line flags.
- Loading blockchain data.
- Calling the correct blockchain function.
- Saving updated blockchain data.
- Printing output to the terminal.
- Handling errors.

The actual blockchain logic is mainly inside the `chain`, `block`, `transaction`, and `storage` packages.

12. Explanation of Commands

12.1 `init` Command

Command:

```
go run ./cmd/toychain init
```

Purpose:

Creates a new blockchain.

What happens internally:

1. A new blockchain object is created.
2. A genesis block is created.
3. The blockchain is saved into `data/chain.json`.
4. The program prints blockchain details.

Example output:

```
New blockchain created
File: data/chain.json
Difficulty: 2
Block size: 5
Genesis hash: 00abc...
```

12.2 `add` Command

Command:

```
go run ./cmd/toychain add -from FAUCET -to Alice -amount 100
```

Purpose:

Adds a transaction to the pending pool.

What happens internally:

1. The blockchain is loaded from the JSON file.
2. A new transaction is created.
3. The transaction is validated.
4. The transaction is added to the pending pool.
5. The blockchain is saved again.

Example output:

```
Transaction added to pending pool: FAUCET -> Alice amount 100
Pending transactions: 1
```

12.3 `pending` Command

Command:

```
go run ./cmd/toychain pending
```

Purpose:

Displays all transactions waiting to be mined.

Example output:

```
Pending transactions  
0. FAUCET -> Alice : 100
```

12.4 mine Command

Command:

```
go run ./cmd/toychain mine
```

Purpose:

Mines pending transactions into a block.

What happens internally:

1. The blockchain is loaded.
2. Pending transactions are selected.
3. A new block is created.
4. The block is linked to the previous block.
5. Proof of work is performed.
6. The new block is added to the blockchain.
7. Mined transactions are removed from the pending pool.
8. The blockchain is saved.

Example output:

```
Block mined successfully  
Height: 1  
Nonce: 2154  
Hash: 00a93f...  
Hashes tried: 2155  
Time taken: 32 ms  
Remaining pending transactions: 0
```

12.5 `print` Command

Command:

```
go run ./cmd/toychain print
```

Purpose:

Prints the full blockchain.

Example output:

```
Blockchain
Difficulty: 2
Block size: 5
Blocks: 2

-----
Height: 0
Timestamp: 2026-07-08T10:00:00
Previous hash:
Nonce: 0
Hash: 00abc...
Transactions:
  none

-----
Height: 1
Timestamp: 2026-07-08T10:05:00
Previous hash: 00abc...
Nonce: 2154
Hash: 00a93f...
Transactions:
  FAUCET -> Alice : 100
```

12.6 `balances` Command

Command:

```
go run ./cmd/toychain balances
```

Purpose:

Shows confirmed account balances.

Example:

```
Balances
Alice: 100
FAUCET: -100
```

The balance is calculated by reading all mined transactions in the blockchain.

If pending transactions should also be included, the following command can be used:

```
go run ./cmd/toychain balances -pending
```

12.7 **validate** Command

Command:

```
go run ./cmd/toychain validate
```

Purpose:

Checks whether the blockchain is valid.

Example output for valid chain:

```
Chain valid
```

Example output for invalid chain:

```
Chain invalid
First offending block: 1
Reason: block hash does not match data
```

12.8 **tamper** Command

Command:

```
go run ./cmd/toychain tamper -block 1 -tx 0 -amount 999
```

Purpose:

Deliberately changes old block data without recalculating the hash.

What happens internally:

1. The blockchain is loaded.
2. The selected block is found.
3. The selected transaction is found.
4. The transaction amount is changed.
5. The blockchain is saved.
6. The hash is not recalculated.

After this, validation should fail.

This proves that blockchain can detect tampering.

13. Data Flow of the System

13.1 Add Transaction Flow

```
graph TD
    A[User enters add command] --> B[CLI reads from, to, amount]
    B --> C[Blockchain is loaded from JSON]
    C --> D[Transaction object is created]
    D --> E[Transaction is added to pending pool]
    E --> F[Blockchain is saved back to JSON]
    F --> G[Success message is printed]
```

13.2 Mining Flow

```
graph TD
    A[User enters mine command] --> B[Blockchain is loaded]
    B --> C[Pending transactions are selected]
```

```
New block is created
  |
  v
Previous hash is attached
  |
  v
Nonce is changed repeatedly
  |
  v
Valid hash is found
  |
  v
Block is added to chain
  |
  v
Pending pool is updated
  |
  v
Blockchain is saved
```

13.3 Validation Flow

```
User enters validate command
  |
  v
Blockchain is loaded
  |
  v
Each block hash is recalculated
  |
  v
Previous hash links are checked
  |
  v
Proof of work is checked
  |
  v
Validation result is printed
```

14. Testing and Validation

The project was formatted, checked, and tested using Go commands.

14.1 Code Formatting

Command:

```
gofmt -w .
```

Purpose:

Formats all Go files according to Go standards.

14.2 Static Checking

Command:

```
go vet ./...
```

Purpose:

Checks for suspicious code patterns and possible mistakes.

14.3 Unit Testing

Command:

```
go test ./...
```

Observed result:

```
ok      toy-blockchain/block      0.484s
ok      toy-blockchain/chain      0.910s
?       toy-blockchain/cmd/toychain [no test files]
?       toy-blockchain/internal/transaction [no test files]
?       toy-blockchain/ledger     [no test files]
ok      toy-blockchain/storage     1.420s
```

This shows that the main tested packages passed successfully.

15. Example Usage

Step 1: Initialize the Blockchain

```
go run ./cmd/toychain init -difficulty 2
```

Expected result:

```
New blockchain created
File: data/chain.json
Difficulty: 2
Block size: 5
Genesis hash: ...
```

Step 2: Add First Transaction

```
go run ./cmd/toychain add -from FAUCET -to Alice -amount 100
```

Expected result:

```
Transaction added to pending pool: FAUCET -> Alice amount 100
Pending transactions: 1
```

Step 3: View Pending Transactions

```
go run ./cmd/toychain pending
```

Expected result:

```
Pending transactions
0. FAUCET -> Alice : 100
```

Step 4: Mine the Transaction

```
go run ./cmd/toychain mine
```

Expected result:

```
Block mined successfully
Height: 1
Nonce: ...
Hash: ...
Hashes tried: ...
```

```
Time taken: ... ms  
Remaining pending transactions: 0
```

Step 5: Add Another Transaction

```
go run ./cmd/toychain add -from Alice -to Bob -amount 30
```

Step 6: Mine Again

```
go run ./cmd/toychain mine
```

Step 7: Check Balances

```
go run ./cmd/toychain balances
```

Expected result:

```
Balances  
Alice: 70  
Bob: 30  
FAUCET: -100
```

Step 8: Validate the Blockchain

```
go run ./cmd/toychain validate
```

Expected result:

```
Chain valid
```

Step 9: Tamper with Old Data

```
go run ./cmd/toychain tamper -block 1 -tx 0 -amount 999
```

Expected result:

```
Tampered block 1 transaction 0 amount: 100 -> 999  
Important: hash was not recalculated, so validation should fail now.
```

Step 10: Validate Again

```
go run ./cmd/toychain validate
```

Expected result:

```
Chain invalid  
First offending block: 1  
Reason: ...
```

This confirms that the blockchain detects data modification.

16. Results and Observations

Observation 1: Transactions Are First Added to the Pending Pool

When the **add** command is executed, the transaction is not directly inserted into the blockchain. It is added to the pending transaction pool.

This is similar to real blockchain systems, where transactions wait before being included in a mined block.

Observation 2: Mining Converts Pending Transactions into Confirmed Transactions

When the **mine** command is executed, pending transactions are moved into a new block. After mining, the pending transaction count decreases.

This shows the difference between unconfirmed and confirmed transactions.

Observation 3: Each Block Is Linked to the Previous Block

Each newly mined block stores the hash of the previous block.

Because of this, the blockchain becomes a linked structure.

If one old block is modified, the link between blocks becomes invalid.

Observation 4: Proof of Work Requires Multiple Hash Attempts

During mining, the system tries different nonce values until a valid hash is found.

The output shows:

```
Hashes tried: ...  
Time taken: ... ms
```

This demonstrates that mining requires computational effort.

Observation 5: Higher Difficulty Increases Mining Work

When the difficulty value is increased, mining usually takes more time because the system must search longer to find a valid hash.

Example:

```
go run ./cmd/toychain init -difficulty 3
```

A difficulty of 3 is harder than a difficulty of 2.

Observation 6: Balance Is Calculated from Transactions

The system does not store account balances directly as fixed values. Instead, balances are calculated from transactions in the blockchain.

For example:

```
FAUCET -> Alice : 100  
Alice -> Bob : 30
```

Then:

```
Alice = 70  
Bob = 30  
FAUCET = -100
```

Observation 7: Tampering Is Detected by Validation

When an old transaction amount is changed using the `tamper` command, the stored hash no longer matches the block data.

Therefore, validation fails.

This is one of the most important results of the project.

17. Limitations

Although this project demonstrates blockchain concepts clearly, it has several limitations.

1. It is not a distributed blockchain.
2. It does not include networking between nodes.
3. It does not use public/private key cryptography.
4. It does not include digital signatures.
5. It does not prevent a user from spending more than their balance.
6. It does not include mining rewards.
7. It does not include transaction fees.
8. It does not include a real consensus mechanism.
9. It stores data in a JSON file instead of a database.
10. It is not suitable for real financial use.

These limitations are acceptable because the project is designed for learning and demonstration.

18. Future Improvements

The project can be improved in several ways.

18.1 Add Digital Signatures

Each transaction can be signed using a private key and verified using a public key.

This would make the system more secure.

18.2 Add Wallets

The project can include wallet generation for users.

A wallet can contain:

- Public key.
 - Private key.
 - Address.
-

18.3 Add Balance Validation

Currently, simple transactions can create negative balances.

A future improvement is to check whether the sender has enough balance before allowing the transaction.

18.4 Add Mining Reward

A reward can be given to the miner after successfully mining a block.

Example:

```
SYSTEM -> Miner : 10
```

18.5 Add Merkle Root

A Merkle root can be added to each block to improve transaction integrity checking.

18.6 Add Peer-to-Peer Networking

Multiple nodes can be connected so that each node keeps a copy of the blockchain.

18.7 Add REST API

A REST API can be created using Go so that external applications can interact with the blockchain.

Example endpoints:

```
POST /transactions  
POST /mine  
GET /chain  
GET /balances  
GET /validate
```

18.8 Add Web Interface

A simple frontend can be created to display blocks, transactions, balances, and validation results visually.

18.9 Add Database Storage

Instead of storing the blockchain in a JSON file, a database such as SQLite, PostgreSQL, or MongoDB can be used.

18.10 Improve Error Messages

The CLI can be improved with clearer error messages and better help instructions for each command.

19. Security Discussion

The most important security concept demonstrated by this project is tamper detection.

In a blockchain, every block depends on the previous block hash. If someone changes a transaction in an old block, the hash of that block changes. Then the next block's previous hash becomes incorrect.

This project proves that concept using the `tamper` command.

Before tampering:

```
Chain valid
```

After tampering:

```
Chain invalid
```

This shows that blockchain does not simply hide data modification. Instead, it makes unauthorized modification visible and detectable.

20. Conclusion

This project successfully implements a basic blockchain command line application using Go. It demonstrates the core blockchain concepts in a simple way.

The application supports creating a blockchain, adding transactions, mining blocks, printing blockchain data, validating the chain, checking balances, viewing pending transactions, and testing tampering.

The most important learning outcome of this project is understanding how blocks are connected using hashes and how blockchain validation can detect changes to old data. The proof-of-work mining process also shows how computational effort is used to create valid blocks.

Although this project is not a real cryptocurrency system, it provides a strong foundation for understanding blockchain internals. It can be further improved by adding wallets, digital signatures, mining rewards, networking, APIs, and a user interface.

Overall, this project is a successful educational implementation of a toy blockchain system.

Appendix A: Main Commands

Command	Purpose
<code>init</code>	Create a new blockchain

Command	Purpose
<code>add</code>	Add a transaction to pending pool
<code>mine</code>	Mine pending transactions into a block
<code>print</code>	Print the full blockchain
<code>validate</code>	Validate the blockchain
<code>balances</code>	Show account balances
<code>pending</code>	Show pending transactions
<code>tamper</code>	Modify old data for testing

Appendix B: Sample Command Sequence

```
go run ./cmd/toychain init -difficulty 2

go run ./cmd/toychain add -from FAUCET -to Alice -amount 100

go run ./cmd/toychain pending

go run ./cmd/toychain mine

go run ./cmd/toychain add -from Alice -to Bob -amount 30

go run ./cmd/toychain mine

go run ./cmd/toychain balances

go run ./cmd/toychain print

go run ./cmd/toychain validate

go run ./cmd/toychain tamper -block 1 -tx 0 -amount 999

go run ./cmd/toychain validate
```

Appendix C: Testing Commands

```
gofmt -w .

go vet ./...

go test ./...
```

Appendix D: Final Project Summary

The final project contains the following completed features:

- Blockchain initialization.
- Genesis block generation.
- Transaction creation.
- Pending transaction pool.
- Mining with proof of work.
- Block hash generation.
- Previous hash linking.
- Blockchain validation.
- Balance calculation.
- JSON file persistence.
- Tamper experiment.
- Unit testing.

This confirms that the project meets the main objectives of a beginner-level blockchain implementation.